

Projeto Final: API de Banco Digital

Turma: LionsDev

Tópicos: Express.js, MongoDB, Mongoose, arquitetura em camadas, bcrypt, JWT, middlewares, autorização, regras de negócio, transações financeiras simuladas, variáveis de ambiente e deploy com Render.

1. Contexto

Você foi contratado para construir a API do **LionsBank**, um banco digital fictício.

A API precisa permitir cadastro de clientes, login, abertura de contas, movimentações financeiras, PIX, boletos, cartões, empréstimos e telas administrativas para gerente.

Importante: este projeto é uma simulação didática. Não existe integração com banco real, dinheiro real, Open Finance, Banco Central, adquirentes de cartão ou sistemas externos.

2. Regras Gerais do Projeto

1. A API deve usar Express, MongoDB, Mongoose, bcryptjs e JWT.
 2. O projeto deve seguir arquitetura em camadas: **routes**, **controllers**, **services**, **repositories**, **models**, **middlewares**, **config** e **utils**.
 3. Toda senha deve ser salva como hash, nunca como texto puro.
 4. Valores monetários devem ser salvos em centavos, usando número inteiro. Exemplo: R\$ 10,50 deve virar **1050**.
 5. Rotas protegidas devem exigir **Authorization: Bearer TOKEN**.
 6. Rotas administrativas devem exigir papel de **gerente** ou **admin**.
 7. Uma pessoa só pode acessar contas, cartões, faturas, boletos e empréstimos dela mesma, exceto gerente/admin.
 8. Operações financeiras devem gerar registro de transação e extrato.
 9. A API nunca deve retornar **senhaHash**, **senhaCartaoHash**, tokens secretos ou dados sensíveis desnecessários.
 10. O projeto deve ter **.env.example**, **README.md**, **requests.http** ou coleção do Postman, e deploy no Render.
-

3. Estrutura Obrigatória

Crie a estrutura abaixo:

```
projeto/
├── src/
│   ├── app.js
│   ├── server.js
│   ├── config/
│   │   └── database.js
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   ├── usuario.controller.js
│   │   ├── conta.controller.js
│   │   ├── transacao.controller.js
│   │   ├── pix.controller.js
│   │   ├── boleto.controller.js
│   │   ├── cartao.controller.js
│   │   ├── emprestimo.controller.js
│   │   └── admin.controller.js
│   ├── middlewares/
│   │   ├── autenticacao.middleware.js
│   │   ├── autorizacao.middleware.js
│   │   ├── validarCampos.middleware.js
│   │   └── erro.middleware.js
│   ├── models/
│   │   ├── usuario.model.js
│   │   ├── conta.model.js
│   │   ├── transacao.model.js
│   │   ├── chavePix.model.js
│   │   ├── boleto.model.js
│   │   ├── cartao.model.js
│   │   ├── fatura.model.js
│   │   └── emprestimo.model.js
│   ├── repositories/
│   ├── routes/
│   ├── services/
│   └── utils/
│       ├── criarErro.js
│       ├── gerarNumeroConta.js
│       ├── dinheiro.js
│       └── gerarCodigo.js
├── .env
├── .env.example
├── package.json
├── render.yaml
└── requests.http
```

4. Variáveis de Ambiente

```
PORT=3000
MONGO_URI=mongodb+srv://usuario:senha@cluster.mongodb.net/lionsbank
JWT_SECRET=troque_essa_chave_por_uma_chave_grande
JWT_EXPIRES_IN=1d
BCRYPT_SALT_ROUNDS=10
LIMITE_PIX_DIARIO_CENTAVOS=500000
```

No Render, não configure **PORT**. A plataforma define essa variável automaticamente.

5. Perfis de Usuário

A API deve trabalhar com três papéis:

Papel	Permissões principais
cliente	Acessar os próprios dados, contas e operações financeiras
gerente	Consultar clientes, aprovar contas, cartões e empréstimos
admin	Tudo que gerente faz e também bloquear/desbloquear usuários

Todo usuário novo deve nascer como **cliente**, exceto usuários criados manualmente no banco para testes administrativos.

6. Models Obrigatórios

6.1 Usuario

Campos mínimos:

- **nome**: obrigatório, mínimo 2 caracteres.
- **email**: obrigatório, único, lowercase.
- **cpf**: obrigatório, único.
- **telefone**: obrigatório.
- **senhaHash**: obrigatório, **select: false**.
- **papel**: **cliente**, **gerente** ou **admin**.
- **status**: **ativo**, **bloqueado** ou **pendente_verificacao**.
- **timestamps**: **true**.

6.2 Conta

Campos mínimos:

- **usuarioId** : referência para **Usuario** .
- **agencia** : texto, exemplo **0001** .
- **numero** : único, gerado pela API.
- **tipo** : **corrente** , **poupanca** ou **salario** .
- **saldoCentavos** : número inteiro, começa em **0** .
- **limiteChequeEspecialCentavos** : número inteiro, começa em **0** .
- **status** : **ativa** , **bloqueada** , **encerrada** ou **pendente_aprovacao** .
- **timestamps** : **true** .

6.3 Transacao

Campos mínimos:

- **contaOrigemId** : referência para **Conta** , opcional em depósito.
- **contaDestinoId** : referência para **Conta** , opcional em saque.
- **tipo** : **deposito** , **saque** , **transferencia** , **pix** , **boleto** , **cartao** , **emprestimo** , **tarifa** ou **estorno** .
- **valorCentavos** : número inteiro maior que zero.
- **descricao** : texto.
- **status** : **pendente** , **aprovada** , **recusada** ou **estornada** .
- **saldoAntesOrigemCentavos** e **saldoDepoisOrigemCentavos** .
- **saldoAntesDestinoCentavos** e **saldoDepoisDestinoCentavos** .
- **timestamps** : **true** .

6.4 ChavePix

Campos mínimos:

- **usuarioId** : referência para **Usuario** .
- **contaId** : referência para **Conta** .
- **tipo** : **cpf** , **email** , **telefone** ou **aleatoria** .
- **valor** : único.
- **ativa** : boolean.
- **timestamps** : **true** .

6.5 Boleto

Campos mínimos:

- **usuarioId** : referência para **Usuario** .
- **contaId** : referência para **Conta** .
- **codigoBarras** : único.
- **beneficiario** : texto.

- **valorCentavos** : número inteiro maior que zero.
- **dataVencimento** : data.
- **status** : **aberto** , **pago** , **vencido** ou **cancelado** .
- **timestamps** : **true** .

6.6 Cartao

Campos mínimos:

- **usuarioId** : referência para **Usuario** .
- **contaId** : referência para **Conta** .
- **tipo** : **debito** ou **credito** .
- **numeroMascarado** : exemplo ****** * 1234** .
- **ultimos4** : texto.
- **senhaCartaoHash** : obrigatório, **select**: **false** .
- **limiteCentavos** : número inteiro.
- **limiteDisponivelCentavos** : número inteiro.
- **status** : **solicitado** , **ativo** , **bloqueado** ou **cancelado** .
- **timestamps** : **true** .

6.7 Fatura

Campos mínimos:

- **cartaoId** : referência para **Cartao** .
- **mesReferencia** : texto, exemplo **2026-06** .
- **valorTotalCentavos** : número inteiro.
- **valorPagoCentavos** : número inteiro.
- **status** : **aberta** , **fechada** , **paga** ou **atrasada** .
- **compras** : lista com descrição, valor, data e estabelecimento.
- **timestamps** : **true** .

6.8 Empréstimo

Campos mínimos:

- **usuarioId** : referência para **Usuario** .
- **contaId** : referência para **Conta** .
- **valorSolicitadoCentavos** : número inteiro.
- **quantidadeParcelas** : número inteiro.
- **taxaJurosMensal** : número.
- **valorParcelaCentavos** : número inteiro.
- **status** : **solicitado** , **aprovado** , **reprovado** , **quitado** ou **em_atraso** .
- **parcelas** : lista com número, valor, vencimento, status e data de pagamento.

- `timestamps: true`.

7. Rotas Obrigatórias

7.1 Autenticação

Método	Rota	Proteção	Descrição
POST	<code>/api/auth/cadastro</code>	Pública	Cadastrar cliente
POST	<code>/api/auth/login</code>	Pública	Fazer login e retornar JWT
POST	<code>/api/auth/logout</code>	JWT	Simular logout do cliente

O logout pode apenas retornar uma mensagem de sucesso. Não precisa implementar blacklist de token.

7.2 Usuários

Método	Rota	Proteção	Descrição
GET	<code>/api/usuarios/perfil</code>	JWT	Ver perfil logado
PATCH	<code>/api/usuarios/perfil</code>	JWT	Atualizar nome, telefone e senha
DELETE	<code>/api/usuarios/perfil</code>	JWT	Encerrar cadastro do cliente
GET	<code>/api/usuarios</code>	Gerente/Admin	Listar usuários
PATCH	<code>/api/usuarios/:id/status</code>	Admin	Bloquear ou desbloquear usuário

7.3 Contas

Método	Rota	Proteção	Descrição
POST	<code>/api/contas</code>	JWT	Solicitar abertura de conta
GET	<code>/api/contas/minhas</code>	JWT	Listar minhas contas
GET	<code>/api/contas/:id</code>	Dono/Admin	Ver detalhes de uma conta
PATCH	<code>/api/contas/:id/status</code>	Gerente/Admin	Aprovar, bloquear ou encerrar
GET	<code>/api/contas/:id/extrato</code>	Dono/Admin	Ver extrato com filtros por data

7.4 Operações Financeiras

Método	Rota	Proteção	Descrição
POST	<code>/api/transacoes/deposito</code>	JWT	Depositar em uma conta

Método	Rota	Proteção	Descrição
POST	/api/transacoes/saque	JWT	Sacar de uma conta
POST	/api/transacoes/transferencia	JWT	Transferir entre contas
PATCH	/api/transacoes/:id/aprovar	Gerente/Admin	Aprovar transferência pendente
PATCH	/api/transacoes/:id/recusar	Gerente/Admin	Recusar transferência pendente
GET	/api/transacoes/:id	Dono/Admin	Ver detalhes de uma transação
POST	/api/transacoes/:id/estorno	Admin	Estornar uma transação aprovada

Regras:

1. Não permitir valores menores ou iguais a zero.
2. Não permitir saque ou transferência sem saldo suficiente.
3. Toda movimentação deve alterar saldo e criar uma transação.
4. Para transferência aprovada, a conta de origem perde saldo e a conta de destino ganha saldo.
5. Para este projeto, não precisa tratar requisições duplicadas. Foque primeiro em validar saldo, alterar o saldo das contas e registrar a transação.

7.5 PIX

Método	Rota	Proteção	Descrição
POST	/api/pix/chaves	JWT	Cadastrar chave PIX
GET	/api/pix/chaves	JWT	Listar minhas chaves
DELETE	/api/pix/chaves/:id	JWT	Desativar chave PIX
POST	/api/pix/enviar	JWT	Enviar PIX para uma chave
GET	/api/pix/limites	JWT	Consultar limite diário disponível

Regras:

1. Uma chave PIX ativa deve ser única no sistema.
2. O PIX deve respeitar o limite diário definido no `.env`.
3. PIX para chave inexistente deve retornar `404`.
4. PIX para conta do próprio usuário é permitido, desde que origem e destino sejam contas diferentes.

7.6 Boletos

Método	Rota	Proteção	Descrição
POST	/api/boletos	JWT	Gerar boleto fictício
GET	/api/boletos	JWT	Listar meus boletos
GET	/api/boletos/:id	JWT	Detalhar boleto
POST	/api/boletos/:id/pagar	JWT	Pagar boleto com uma conta
PATCH	/api/boletos/:id/cancelar	JWT	Cancelar boleto em aberto

Regras:

1. Boleto pago deve gerar transação do tipo **boleto**.
2. Não permitir pagar boleto cancelado ou já pago.
3. Não permitir pagamento sem saldo suficiente.

7.7 Cartões

Método	Rota	Proteção	Descrição
POST	/api/cartoes	JWT	Solicitar cartão
GET	/api/cartoes	JWT	Listar meus cartões
PATCH	/api/cartoes/:id/status	Gerente/Admin	Aprovar, bloquear ou cancelar
POST	/api/cartoes/:id/compras	JWT	Registrar compra fictícia
GET	/api/cartoes/:id/fatura	JWT	Consultar fatura atual
POST	/api/cartoes/:id/fatura/pagar	JWT	Pagar fatura

Regras:

1. Compra no débito desconta saldo da conta imediatamente.
2. Compra no crédito consome limite disponível e entra na fatura.
3. Não retornar número completo do cartão.
4. Senha do cartão deve ser salva como hash.

7.8 Empréstimos

Método	Rota	Proteção	Descrição
POST	/api/emprestimos/simular	JWT	Simular parcelas e juros
POST	/api/emprestimos	JWT	Solicitar empréstimo

Método	Rota	Proteção	Descrição
GET	<code>/api/emprestimos</code>	JWT	Listar meus empréstimos
PATCH	<code>/api/emprestimos/:id/aprovar</code>	Gerente/Admin	Aprovar empréstimo
PATCH	<code>/api/emprestimos/:id/reprovar</code>	Gerente/Admin	Reprovar empréstimo
POST	<code>/api/emprestimos/:id/parcelas/:numero/pagar</code>	JWT	Pagar parcela

Regras:

1. Simulação não salva empréstimo no banco.
2. Aprovação deve depositar o valor solicitado na conta do cliente.
3. Pagamento de parcela deve gerar transação.
4. Não permitir pagar a mesma parcela duas vezes.

7.9 Painel Administrativo

Método	Rota	Proteção	Descrição
GET	<code>/api/admin/resumo</code>	Gerente/Admin	Total de usuários, contas e saldo
GET	<code>/api/admin/transacoes</code>	Gerente/Admin	Listar transações com filtros
GET	<code>/api/admin/contas-pendentes</code>	Gerente/Admin	Contas aguardando aprovação
GET	<code>/api/admin/emprestimos-pendentes</code>	Gerente/Admin	Empréstimos aguardando análise

8. Middlewares Obrigatórios

8.1 Autenticação

Deve validar o JWT e adicionar os dados do usuário em `req.usuario`.

8.2 Autorização por Papel

Crie um middleware para permitir somente alguns papéis:

```
autorizar(["gerente", "admin"])
```

8.3 Dono do Recurso

Implemente a regra de que um cliente só pode acessar os próprios recursos.

Exemplo: um cliente não pode consultar extrato de uma conta que pertence a outro cliente.

8.4 Erro Centralizado

Toda exceção deve passar por um middleware de erro e retornar JSON padronizado:

```
{  
  "message": "Mensagem do erro"  
}
```

9. Regras Realistas de Banco

Implemente pelo menos 10 regras abaixo:

1. CPF e email devem ser únicos.
2. Conta nova nasce como **pendente_aprovacao**.
3. Cliente bloqueado não pode movimentar dinheiro.
4. Conta bloqueada não pode sacar, transferir, pagar boleto ou enviar PIX.
5. Saque cobra tarifa fictícia de R\$ 2,50.
6. Poupança não pode ter cheque especial.
7. Conta salário só pode fazer saque e transferência para conta do mesmo CPF.
8. PIX tem limite diário.
9. Transferência acima de R\$ 5.000,00 fica **pendente** até aprovação de gerente.
10. Estorno só pode acontecer uma vez.
11. Cartão de crédito só pode comprar se houver limite disponível.
12. Fatura paga libera limite do cartão.
13. Empréstimo reprovado deve guardar motivo.
14. Parcelas vencidas mudam status para **em_atraso**.
15. Exclusão de usuário deve ser lógica: mude o status para bloqueado ou encerrado, em vez de apagar histórico financeiro.

10. Segurança e Boas Práticas

1. Use mensagens genéricas no login: **Email ou senha incorretos.**
 2. Nunca retorne **senhaHash**.
 3. Nunca retorne **senhaCartaoHash**.
 4. Nunca salve valor monetário com número decimal.
 5. Valide campos obrigatórios antes de chamar o service.
 6. Coloque regra de negócio no service, não no controller.
 7. Coloque acesso ao banco no repository.
 8. Use **runValidators: true** em updates do Mongoose.
 9. Use **select: false** para hashes.
 10. Padronize respostas de erro.
 11. Use **.env** para segredos.
 12. Não suba **.env** para o GitHub.
-

11. Fluxos de Teste Obrigatórios

Teste e documente os fluxos abaixo no **requests.http** ou Postman:

1. Cadastrar cliente.
2. Fazer login.
3. Solicitar abertura de conta.
4. Aprovar conta como gerente/admin.
5. Depositar dinheiro.
6. Sacar dinheiro.
7. Transferir dinheiro entre duas contas.
8. Cadastrar chave PIX.
9. Enviar PIX.
10. Gerar boleto.
11. Pagar boleto.
12. Solicitar cartão.
13. Aprovar cartão.
14. Registrar compra no crédito.
15. Consultar fatura.
16. Pagar fatura.
17. Simular empréstimo.
18. Solicitar empréstimo.
19. Aprovar empréstimo.
20. Pagar parcela de empréstimo.
21. Bloquear usuário e tentar movimentar dinheiro.
22. Fazer deploy no Render.

Não basta o caminho feliz. Para cada regra que você implementou na seção 9, inclua também **um teste que prova a recusa** — ex.: sacar sem saldo, cliente bloqueado tentando mover dinheiro, pagar boleto já pago, estornar duas vezes, PIX acima do limite diário. É isso que separa "funciona no print" de "a regra realmente existe".

12. Entregáveis

O aluno deve entregar:

1. Link do repositório no GitHub.
2. Link público do deploy no Render.
3. `README.md` explicando como rodar, variáveis de ambiente e rotas principais.
4. `.env.example` sem segredos reais.
5. `requests.http` ou coleção Postman com todos os fluxos.
6. Prints ou evidências dos testes principais.

Gabarito de referência do professor:

`modulo09/exercicios_resolvidos/api_banco_digital`.

13. Roadmap Sugerido

Esta é **uma ordem sugerida, não uma receita**: você decide a granularidade e pode reorganizar. O que importa é o **portão de cada fase** — só siga em frente quando ele passar. O *o quê* de cada fase está nas seções 6 a 9; aqui ficam os critérios para você se considerar pronto.

Fase 1 — Base

Pronto quando: cadastro e login funcionam, a senha vai pro banco como hash, rotas protegidas barram quem não manda `Bearer` token (`401`) e `req.usuario` chega nos controllers já com `papel`.

Fase 2 — Contas

Pronto quando: uma conta nasce `pendente_aprovacao`, gerente/admin consegue aprovar, e um cliente só enxerga as próprias contas (pedir a conta de outro → `403` / `404`).

Fase 3 — Movimentações

Pronto quando: depósito, saque e transferência alteram o saldo e geram transação com extrato; saque/transferência sem saldo → erro; transferência aprovada tira da origem e põe no destino; todo valor em centavos (inteiro).

Fase 4 — Produtos Bancários

Pronto quando: PIX respeita o limite diário e a unicidade da chave; boleto não é pago duas vezes; cartão de crédito só compra dentro do limite e a compra entra na fatura; empréstimo aprovado deposita o valor e gera as parcelas.

Fase 5 — Administração e Deploy

Pronto quando: gerente/admin veem os painéis, admin bloqueia usuário (e cliente bloqueado não move dinheiro), a exclusão é lógica (não apaga histórico financeiro) e a API está no ar no Render.

14. Critérios de Avaliação

Critério	Pontos
Estrutura em camadas organizada	10
Autenticação com bcrypt e JWT	10
Models bem definidos no Mongoose	10
Regras de negócio financeiras	20
Controle de permissões e dono do recurso	10
Transações e extrato coerentes	10
PIX, boletos, cartões e empréstimos	15
Tratamento de erros e validações	5
Documentação e coleção de testes	5
Deploy funcionando no Render	5

Total: **100 pontos**.

15. Desafios Extras

Para quem quiser ir além:

1. Implementar refresh token.
2. Implementar recuperação de senha com token temporário.

3. Criar paginação em listagens grandes.
 4. Criar filtros por período no extrato.
 5. Criar log de auditoria para toda ação administrativa.
 6. Usar transações do MongoDB com session para operações financeiras.
 7. Criar testes automatizados com Jest ou Vitest.
 8. Criar documentação Swagger/OpenAPI.
 9. Criar um script opcional para gerar gerente, clientes, contas e transações automaticamente.
 10. Criar um frontend simples para consumir a API.
 11. Criar uma regra extra para evitar que a mesma operação financeira seja cadastrada duas vezes.
-

LionsDev • Professor Nicolas Cardoso Motta
Projeto Final de API de Banco Digital - Módulo 09